



# SpaceX Falcon 9 first stage Landing Prediction

## Lab 1: Collecting the data

Estimated time needed: **45** minutes

In this capstone, we will predict if the Falcon 9 first stage will land successfully. SpaceX advertises Falcon 9 rocket launches on its website with a cost of 62 million dollars; other providers cost upward of 165 million dollars each, much of the savings is because SpaceX can reuse the first stage. Therefore if we can determine if the first stage will land, we can determine the cost of a launch. This information can be used if an alternate company wants to bid against SpaceX for a rocket launch. In this lab, you will collect and make sure the data is in the correct format from an API. The following is an example of a successful and launch.



Several examples of an unsuccessful landing are shown here:



Most unsuccessful landings are planned. Space X performs a controlled landing in the oceans.

## Objectives

In this lab, you will make a get request to the SpaceX API. You will also do some basic data wrangling and formatting.

- Request to the SpaceX API
- Clean the requested data

## Import Libraries and Define Auxiliary Functions

We will import the following libraries into the lab

```
In [76]: # Requests allows us to make HTTP requests which we will use to get data from an API
import requests
# Pandas is a software library written for the Python programming language for data manipulation
import pandas as pd
# NumPy is a library for the Python programming language, adding support for large, multi-dimensional arrays and matrices, along with a large library of high-level mathematical functions
import numpy as np
# Datetime is a library that allows us to represent dates
import datetime

# Setting this option will print all columns of a dataframe
pd.set_option('display.max_columns', None)
# Setting this option will print all of the data in a feature
pd.set_option('display.max_colwidth', None)
```

Below we will define a series of helper functions that will help us use the API to extract information using identification numbers in the launch data.

From the `rocket` column we would like to learn the booster name.

```
In [77]: # Takes the dataset and uses the rocket column to call the API and append the data to the list
def getBoosterVersion(data):
    for x in data['rocket']:
        if x:
            response = requests.get("https://api.spacexdata.com/v4/rockets/"+str(x)).json()
            BoosterVersion.append(response['name'])
```

From the `launchpad` we would like to know the name of the launch site being used, the longitude, and the latitude.

```
In [78]: # Takes the dataset and uses the launchpad column to call the API and append the data to the list
def getLaunchSite(data):
    for x in data['launchpad']:
        if x:
            response = requests.get("https://api.spacexdata.com/v4/launchpads/"+str(x)).json()
            Longitude.append(response['longitude'])
            Latitude.append(response['latitude'])
            LaunchSite.append(response['name'])
```

From the `payload` we would like to learn the mass of the payload and the orbit that it is going to.

```
In [79]: # Takes the dataset and uses the payloads column to call the API and append the data to the list
def getPayloadData(data):
    for load in data['payloads']:
        if load:
            response = requests.get("https://api.spacexdata.com/v4/payloads/"+load).json()
            PayloadMass.append(response['mass_kg'])
            Orbit.append(response['orbit'])
```

From `cores` we would like to learn the outcome of the landing, the type of the landing, number of flights with that core, whether gridfins were used, whether the core is reused, whether legs were used, the landing pad used, the block of the core which is a number used to separate version of cores, the number of times this specific core has been reused, and the serial of the core.

```
In [80]: # Takes the dataset and uses the cores column to call the API and append the data to the lists
def getCoreData(data):
    for core in data['cores']:
        if core['core'] != None:
            response = requests.get("https://api.spacexdata.com/v4/cores/"+core['core']).json()
            Block.append(response['block'])
            ReusedCount.append(response['reuse_count'])
            Serial.append(response['serial'])
        else:
            Block.append(None)
            ReusedCount.append(None)
```

```

Serial.append(None)
Outcome.append(str(core['landing_success'])+' '+str(core['landing_type']))
Flights.append(core['flight'])
GridFins.append(core['gridfins'])
Reused.append(core['reused'])
Legs.append(core['legs'])
LandingPad.append(core['landpad'])

```

Now let's start requesting rocket launch data from SpaceX API with the following URL:

```
In [81]: spacex_url="https://api.spacexdata.com/v4/launches/past"
```

```
In [82]: response = requests.get(spacex_url)
```

Check the content of the response

```
In [83]: # print(response.content)
print(response.content[:800])
```

```

b' [{"fairings": {"reused": false, "recovery_attempt": false, "recovered": false, "ships": []}, "links":
{"patch": {"small": "https://images2.imgbox.com/94/f2/NN6Ph45r_o.png", "large": "https://images2.img
box.com/5b/02/QcxHUb5V_o.png"}, "reddit": {"campaign": null, "launch": null, "media": null, "recovery": n
ull}, "flickr": {"small": [], "original": []}, "presskit": null, "webcast": "https://www.youtube.com/watc
h?v=0a_00nJ_Y88", "youtube_id": "0a_00nJ_Y88", "article": "https://www.space.com/2196-spacex-inaugur
al-falcon-1-rocket-lost-launch.html", "wikipedia": "https://en.wikipedia.org/wiki/DemoSat"}, "stati
c_fire_date_utc": "2006-03-17T00:00:00.000Z", "static_fire_date_unix": 1142553600, "net": false, "wind
ow": 0, "rocket": "5e9d0d95eda69955f709d1eb", "success": false, "failures": [{"time": 33, "altitude": nul
l, "reason": "merlin engine failure"}], '

```

You should see the response contains massive information about SpaceX launches. Next, let's try to discover some more relevant information for this project.

## Task 1: Request and parse the SpaceX launch data using the GET request

To make the requested JSON results more consistent, we will use the following static response object for this project:

```
In [84]: static_json_url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DS0321E'
```

We should see that the request was successful with the 200 status response code

```
In [85]: response=requests.get(static_json_url)
```

```
In [86]: response.status_code
```

```
Out[86]: 200
```

Now we decode the response content as a Json using `.json()` and turn it into a Pandas dataframe using `.json_normalize()`

```
In [87]: # Use json_normalize meethod to convert the json result into a dataframe

# Decode JSON response
data1 = response.json()

# Normalise into a dataframe
data = pd.json_normalize(data1)
```

Using the dataframe `data` print the first 5 rows

```
In [88]: # Get the head of the dataframe

data.head()
```

Out [88]:

|   | static_fire_date_utc     | static_fire_date_unix | tbd   | net   | window | rocket                   | success |                                            |
|---|--------------------------|-----------------------|-------|-------|--------|--------------------------|---------|--------------------------------------------|
| 0 | 2006-03-17T00:00:00.000Z | 1.142554e+09          | False | False | 0.0    | 5e9d0d95eda69955f709d1eb | False   | ar                                         |
| 1 | None                     | NaN                   | False | False | 0.0    | 5e9d0d95eda69955f709d1eb | False   | P<br>s<br>a<br><br>f                       |
| 2 | None                     | NaN                   | False | False | 0.0    | 5e9d0d95eda69955f709d1eb | False   | f<br><br>t<br>st                           |
| 3 | 2008-09-20T00:00:00.000Z | 1.221869e+09          | False | False | 0.0    | 5e9d0d95eda69955f709d1eb | True    | R<br>e<br><br>si<br><br>fu<br>de<br><br>th |
| 4 | None                     | NaN                   | False | False | 0.0    | 5e9d0d95eda69955f709d1eb | True    |                                            |

In [89]: data.describe()

Out[89]:

|       | static_fire_date_unix | window       | flight_number | date_unix    | fairings |
|-------|-----------------------|--------------|---------------|--------------|----------|
| count | 1.000000e+02          | 100.000000   | 107.000000    | 1.070000e+02 | 0.0      |
| mean  | 1.499305e+09          | 2553.900000  | 54.000000     | 1.494051e+09 | NaN      |
| std   | 8.414133e+07          | 4108.909732  | 31.032241     | 9.692164e+07 | NaN      |
| min   | 1.142554e+09          | 0.000000     | 1.000000      | 1.143239e+09 | NaN      |
| 25%   | 1.465406e+09          | 0.000000     | 27.500000     | 1.458641e+09 | NaN      |
| 50%   | 1.516898e+09          | 0.000000     | 54.000000     | 1.517434e+09 | NaN      |
| 75%   | 1.556707e+09          | 5595.000000  | 80.500000     | 1.560891e+09 | NaN      |
| max   | 1.605111e+09          | 18300.000000 | 107.000000    | 1.605486e+09 | NaN      |

In [90]: `data.columns.values`

Out[90]: array(['static\_fire\_date\_utc', 'static\_fire\_date\_unix', 'tbd', 'net',  
'window', 'rocket', 'success', 'details', 'crew', 'ships',  
'capsules', 'payloads', 'launchpad', 'auto\_update', 'failures',  
'flight\_number', 'name', 'date\_utc', 'date\_unix', 'date\_local',  
'date\_precision', 'upcoming', 'cores', 'id', 'fairings.reused',  
'fairings.recovery\_attempt', 'fairings.recovered',  
'fairings.ships', 'links.patch.small', 'links.patch.large',  
'links.reddit.campaign', 'links.reddit.launch',  
'links.reddit.media', 'links.reddit.recovery',  
'links.flickr.small', 'links.flickr.original', 'links.presskit',  
'links.webcast', 'links.youtube\_id', 'links.article',  
'links.wikipedia', 'fairings'], dtype=object)

You will notice that a lot of the data are IDs. For example the rocket column has no information about the rocket just an identification number.

We will now use the API again to get information about the launches using the IDs given for each launch. Specifically we will be using columns `rocket`, `payloads`, `launchpad`, and `cores`.

In [91]: `# Lets take a subset of our dataframe keeping only the features we want and the flight number,  
data = data[['rocket', 'payloads', 'launchpad', 'cores', 'flight_number', 'date_utc']]`

In [92]: `data.head(2)`

Out[92]:

|   | rocket                   | payloads                   | launchpad                |
|---|--------------------------|----------------------------|--------------------------|
| 0 | 5e9d0d95eda69955f709d1eb | [5eb0e4b5b6c3bb0006eeb1e1] | 5e9e4502f5090995de566f86 |
| 1 | 5e9d0d95eda69955f709d1eb | [5eb0e4b6b6c3bb0006eeb1e2] | 5e9e4502f5090995de566f86 |

In [93]: `# We will remove rows with multiple cores because those are falcon rockets with 2 extra rocket  
data = data[data['cores'].map(len)==1]  
data = data[data['payloads'].map(len)==1]  
  
# Since payloads and cores are lists of size 1 we will also extract the single value in the list  
data['cores'] = data['cores'].map(lambda x : x[0])  
data['payloads'] = data['payloads'].map(lambda x : x[0])  
  
# We also want to convert the date_utc to a datetime datatype and then extracting the date leave  
data['date'] = pd.to_datetime(data['date_utc']).dt.date`

```
# Using the date we will restrict the dates of the launches, all before the cut off date of 13th
data = data[data['date'] <= datetime.date(2020, 11, 13)]
```

- From the `rocket` we would like to learn the booster name
- From the `payload` we would like to learn the mass of the payload and the orbit that it is going to
- From the `launchpad` we would like to know the name of the launch site being used, the longitude, and the latitude.
- From `cores` we would like to learn the outcome of the landing, the type of the landing, number of flights with that core, whether gridfins were used, whether the core is reused, whether legs were used, the landing pad used, the block of the core which is a number used to separate version of cores, the number of times this specific core has been reused, and the serial of the core.

The data from these requests will be stored in lists and will be used to create a new dataframe.

```
In [94]: #Global variables
BoosterVersion = []
PayloadMass = []
Orbit = []
LaunchSite = []
Outcome = []
Flights = []
GridFins = []
Reused = []
Legs = []
LandingPad = []
Block = []
ReusedCount = []
Serial = []
Longitude = []
Latitude = []
```

These functions will apply the outputs globally to the above variables. Let's take a look at `BoosterVersion` variable. Before we apply `getBoosterVersion` the list is empty:

```
In [95]: BoosterVersion
```

```
Out[95]: []
```

Now, let's apply `getBoosterVersion` function method to get the booster version

```
In [96]: # Call the function previously made called getBoosterVersion
getBoosterVersion(data)
```

the list has now been update

```
In [97]: BoosterVersion[0:5]
```

```
Out[97]: ['Falcon 1', 'Falcon 1', 'Falcon 1', 'Falcon 1', 'Falcon 9']
```

we can apply the rest of the functions here:

```
In [98]: # Call getLaunchSite
getLaunchSite(data)
```

```
In [99]: # Call getPayloadData
getPayloadData(data)
```

```
In [100]: # Call getCoreData
getCoreData(data)
```

Finally let's construct our dataset using the data we have obtained. We will combine the columns into a dictionary.

```
In [101]: launch_dict = {'FlightNumber': list(data['flight_number']),
                        'Date': list(data['date']),
                        'BoosterVersion': BoosterVersion,
                        'PayloadMass': PayloadMass,
                        'Orbit': Orbit,
                        'LaunchSite': LaunchSite,
                        'Outcome': Outcome,
                        'Flights': Flights,
                        'GridFins': GridFins,
                        'Reused': Reused,
                        'Legs': Legs,
                        'LandingPad': LandingPad,
                        'Block': Block,
                        'ReusedCount': ReusedCount,
                        'Serial': Serial,
                        'Longitude': Longitude,
                        'Latitude': Latitude}
```

Then, we need to create a Pandas data frame from the dictionary launch\_dict.

```
In [102]: # Create a data from launch_dict

df_launch = pd.DataFrame(launch_dict)
```

Show the summary of the dataframe

```
In [103]: # Show the head of the dataframe

df_launch.head()
```

```
Out[103]:
```

|   | FlightNumber | Date       | BoosterVersion | PayloadMass | Orbit | LaunchSite      | Outcome   | Flights | GridFins | Reu |
|---|--------------|------------|----------------|-------------|-------|-----------------|-----------|---------|----------|-----|
| 0 | 1            | 2006-03-24 | Falcon 1       | 20.0        | LEO   | Kwajalein Atoll | None None | 1       | False    | F   |
| 1 | 2            | 2007-03-21 | Falcon 1       | NaN         | LEO   | Kwajalein Atoll | None None | 1       | False    | F   |
| 2 | 4            | 2008-09-28 | Falcon 1       | 165.0       | LEO   | Kwajalein Atoll | None None | 1       | False    | F   |
| 3 | 5            | 2009-07-13 | Falcon 1       | 200.0       | LEO   | Kwajalein Atoll | None None | 1       | False    | F   |
| 4 | 6            | 2010-06-04 | Falcon 9       | NaN         | LEO   | CCSFS SLC 40    | None None | 1       | False    | F   |

## Task 2: Filter the dataframe to only include Falcon 9 launches

Finally we will remove the Falcon 1 launches keeping only the Falcon 9 launches. Filter the data dataframe using the `BoosterVersion` column to only keep the Falcon 9 launches. Save the filtered data to a new dataframe called `data_falcon9`.

```
In [104]: data.columns
```

```
Out[104]: Index(['rocket', 'payloads', 'launchpad', 'cores', 'flight_number', 'date_utc',
               'date'],
              dtype='object')
```

```
In [105]: df_launch.columns
```

```
Out[105]: Index(['FlightNumber', 'Date', 'BoosterVersion', 'PayloadMass', 'Orbit',
               'LaunchSite', 'Outcome', 'Flights', 'GridFins', 'Reused', 'Legs',
               'LandingPad', 'Block', 'ReusedCount', 'Serial', 'Longitude',
               'Latitude'],
              dtype='object')
```

```
In [106... # Hint data['BoosterVersion']!= 'Falcon 1'
data_falcon9 = df_launch[df_launch['BoosterVersion'] != 'Falcon 1']
```

```
In [107... data_falcon9.shape
```

```
Out[107]: (90, 17)
```

```
In [108... data_falcon9.head()
```

```
Out[108]:
```

|   | FlightNumber | Date       | BoosterVersion | PayloadMass | Orbit | LaunchSite   | Outcome        | Flights | GridFins | Reus |
|---|--------------|------------|----------------|-------------|-------|--------------|----------------|---------|----------|------|
| 4 | 6            | 2010-06-04 | Falcon 9       | NaN         | LEO   | CCSFS SLC 40 | None<br>None   | 1       | False    | Fa   |
| 5 | 8            | 2012-05-22 | Falcon 9       | 525.0       | LEO   | CCSFS SLC 40 | None<br>None   | 1       | False    | Fa   |
| 6 | 10           | 2013-03-01 | Falcon 9       | 677.0       | ISS   | CCSFS SLC 40 | None<br>None   | 1       | False    | Fa   |
| 7 | 11           | 2013-09-29 | Falcon 9       | 500.0       | PO    | VAFB SLC 4E  | False<br>Ocean | 1       | False    | Fa   |
| 8 | 12           | 2013-12-03 | Falcon 9       | 3170.0      | GTO   | CCSFS SLC 40 | None<br>None   | 1       | False    | Fa   |

Now that we have removed some values we should reset the FlightNumber column

```
In [109... data_falcon9.loc[:, 'FlightNumber'] = list(range(1, data_falcon9.shape[0]+1))
data_falcon9
```

/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages/pandas/core/indexing.py:1773: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row\_indexer,col\_indexer] = value instead

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user\\_guide/indexing.html#returning-a-view-versus-a-copy](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)  
self.\_setitem\_single\_column(ilocs[0], value, pi)



Out [109]:

|     | FlightNumber | Date       | BoosterVersion | PayloadMass | Orbit | LaunchSite   | Outcome     | Flights | GridFins | Re  |
|-----|--------------|------------|----------------|-------------|-------|--------------|-------------|---------|----------|-----|
|     | 4            | 2010-06-04 | Falcon 9       | NaN         | LEO   | CCSFS SLC 40 | None None   | 1       | False    |     |
|     | 5            | 2012-05-22 | Falcon 9       | 525.0       | LEO   | CCSFS SLC 40 | None None   | 1       | False    |     |
|     | 6            | 2013-03-01 | Falcon 9       | 677.0       | ISS   | CCSFS SLC 40 | None None   | 1       | False    |     |
|     | 7            | 2013-09-29 | Falcon 9       | 500.0       | PO    | VAFB SLC 4E  | False Ocean | 1       | False    |     |
|     | 8            | 2013-12-03 | Falcon 9       | 3170.0      | GTO   | CCSFS SLC 40 | None None   | 1       | False    |     |
| ... | ...          | ...        | ...            | ...         | ...   | ...          | ...         | ...     | ...      | ... |
|     | 89           | 2020-09-03 | Falcon 9       | 15600.0     | VLEO  | KSC LC 39A   | True ASDS   | 2       | True     |     |
|     | 90           | 2020-10-06 | Falcon 9       | 15600.0     | VLEO  | KSC LC 39A   | True ASDS   | 3       | True     |     |
|     | 91           | 2020-10-18 | Falcon 9       | 15600.0     | VLEO  | KSC LC 39A   | True ASDS   | 6       | True     |     |
|     | 92           | 2020-10-24 | Falcon 9       | 15600.0     | VLEO  | CCSFS SLC 40 | True ASDS   | 3       | True     |     |
|     | 93           | 2020-11-05 | Falcon 9       | 3681.0      | MEO   | CCSFS SLC 40 | True ASDS   | 1       | True     |     |

90 rows x 17 columns

## Data Wrangling

We can see below that some of the rows are missing values in our dataset.

```
In [110]: data_falcon9.isnull().sum()
```

```
Out [110]: FlightNumber    0
Date                    0
BoosterVersion         0
PayloadMass            5
Orbit                  0
LaunchSite             0
Outcome                0
Flights                0
GridFins               0
Reused                 0
Legs                   0
LandingPad            26
Block                  0
ReusedCount            0
Serial                 0
Longitude              0
Latitude               0
dtype: int64
```

Before we can continue we must deal with these missing values. The `LandingPad` column will retain None values to represent when landing pads were not used.

### Task 3: Dealing with Missing Values

Calculate below the mean for the `PayloadMass` using the `.mean()`. Then use the mean and the `.replace()` function to replace `np.nan` values in the data with the mean you calculated.

```
In [111]: data_falcon9.describe()
```

Out[111]:

|              | FlightNumber | PayloadMass  | Flights   | Block     | ReusedCount | Longitude   | Latitude  |
|--------------|--------------|--------------|-----------|-----------|-------------|-------------|-----------|
| <b>count</b> | 90.000000    | 85.000000    | 90.000000 | 90.000000 | 90.000000   | 90.000000   | 90.000000 |
| <b>mean</b>  | 45.500000    | 6123.547647  | 1.788889  | 3.500000  | 3.188889    | -86.366477  | 29.449963 |
| <b>std</b>   | 26.124701    | 4870.916417  | 1.213172  | 1.595288  | 4.194417    | 14.149518   | 2.141306  |
| <b>min</b>   | 1.000000     | 350.000000   | 1.000000  | 1.000000  | 0.000000    | -120.610829 | 28.561857 |
| <b>25%</b>   | 23.250000    | 2482.000000  | 1.000000  | 2.000000  | 0.000000    | -80.603956  | 28.561857 |
| <b>50%</b>   | 45.500000    | 4535.000000  | 1.000000  | 4.000000  | 1.000000    | -80.577366  | 28.561857 |
| <b>75%</b>   | 67.750000    | 9600.000000  | 2.000000  | 5.000000  | 4.000000    | -80.577366  | 28.608058 |
| <b>max</b>   | 90.000000    | 15600.000000 | 6.000000  | 5.000000  | 13.000000   | -80.577366  | 34.632093 |

```
In [112... # Calculate the mean value of PayloadMass column

PayloadMass_mean = data_falcon9['PayloadMass'].mean()
print("PayloadMass_mean = ", PayloadMass_mean)

# Replace the np.nan values with its mean value
data_falcon9['PayloadMass'].fillna(PayloadMass_mean, inplace=True)

PayloadMass_mean = 6123.547647058824

/home/jupyterlab/conda/envs/python/lib/python3.7/site-packages/pandas/core/generic.py:6392: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/in
dexing.html#returning-a-view-versus-a-copy
    return self._update_inplace(result)
```

```
In [113... data_falcon9.isnull().sum()
```

```
Out[113]: FlightNumber      0
         Date              0
         BoosterVersion    0
         PayloadMass       0
         Orbit            0
         LaunchSite        0
         Outcome           0
         Flights           0
         GridFins          0
         Reused            0
         Legs              0
         LandingPad       26
         Block            0
         ReusedCount       0
         Serial           0
         Longitude        0
         Latitude         0
         dtype: int64
```

You should see the number of missing values of the `PayloadMass` change to zero.

Now we should have no missing values in our dataset except for in `LandingPad`.

We can now export it to a **CSV** for the next section, but to make the answers consistent, in the next lab we will provide data in a pre-selected date range.

```
data_falcon9.to_csv('dataset_part_1.csv', index=False)
```

## Authors

[Joseph Santarcangelo](#) has a PhD in Electrical Engineering, his research focused on using machine learning, signal processing, and computer vision to determine how videos impact human cognition. Joseph has been working for IBM since he completed his PhD.

Copyright ©IBM Corporation. All rights reserved.